

《漏洞利用及渗透测试基础》实验报告

姓名：王一诺 学号：2312331 班级：计科二班

实验名称：

程序插桩及 Hook 实验

实验要求：

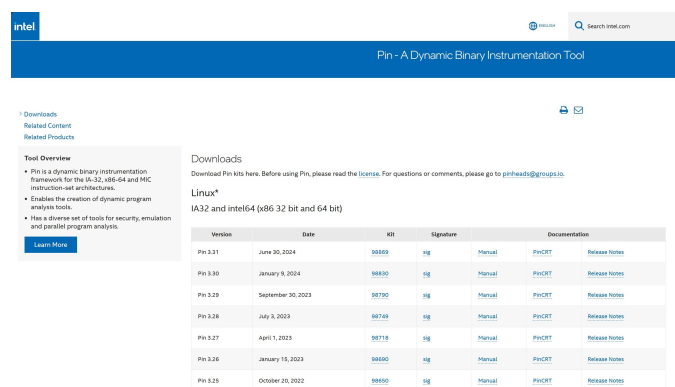
复现实验一，基于 Windows MyPinTool 或在 Kali 中复现 malloctrace 这个 PinTool，理解 Pin 插桩工具的核心步骤和相关 API，关注 malloc 和 free 函数的输入输出信息。

实验过程：

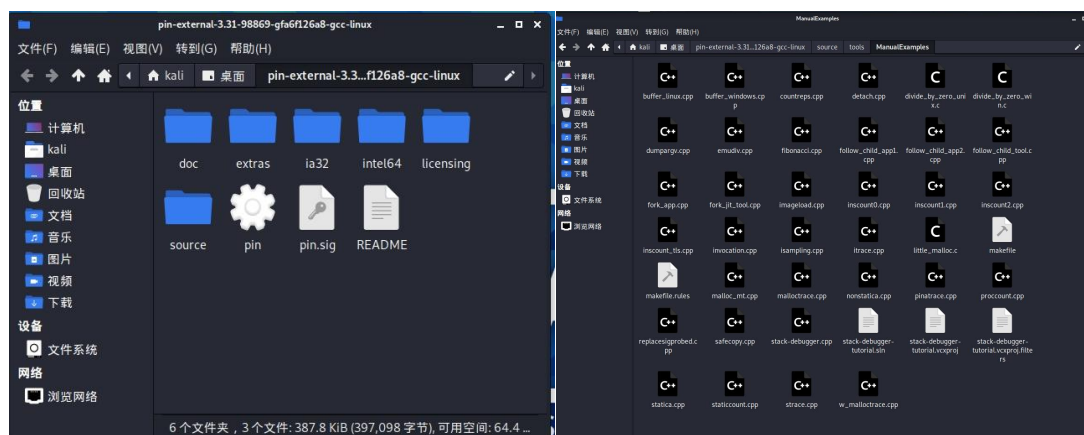
Pin 是 Intel 公司开发的动态二进制插桩框架,支持 IA-32 和 x86-64 指令集架构,支持 windows 和 linux。这里我选择在 kali 虚拟机中安装和使用 Pin。

1. 安装 Pin

从官网下载 linux 版本的 Pin 压缩包，并把压缩包拖入 kali 虚拟机。



在虚拟机中解压并打开，可以看到如左下图所示的文件夹结构：



文件夹 ia32 和 intel64 包含了英特尔不同体系架构下的相关库和可执行文件,文件夹 doc 包含了 Pin 相关的用户手册、API 文档等,而文件夹 source\tools 里包含了大量的 PinTool。打开文件夹 source 中的 tools 中的 ManualExamples，可以查看到很多已经编写好的 Pintool，如右上图所示。

2. 以 inscout0 为例的 Pintool 插桩框架和基本用法

Pin 通过已经定义的 tools 或者自己开发的 tool 来完成对目标程序的插桩。通常, PinTool 以动态链接库方式使用, Linux 下就是 .so 文件。inscout0.cpp 是已经写好的进行指令技术的一个典型指令级插桩, 我们借此分析 pintool 的插桩框架和基本用法。

2. 1>分析插桩框架

```
ofstream OutFile;
static UINT64 icount = 0; //静态变量, 保存运行的指令数的计数
VOID docount() { icount++; } //这个函数在每条指令执行以前被调用
VOID Instruction(INS ins, VOID* v) { //Pin 工具每次遇到一个新指令都会调用该函数
    //在每个指令之前插入一个函数 docount 的调用, 没有任何参数
    INS_InsertCall(ins, IPOINT_BEFORE, (AFUNPTR)docount, IARG_END);
}
//指定输出文件为 inscount.out
KNOB< string > KnobOutputFile(KNOB_MODE_WRITEONCE, "pintool", "o", "inscount.out", "specify
output file name");
VOID Fini(INT32 code, VOID* v) { //当应用退出的时候调用本函数
    OutFile.setf(ios::showbase);
    OutFile << "Count " << icount << endl;
    OutFile.close();
}
int main(int argc, char* argv[]) {
    // 调用函数 PIN_Init 完成初始化
    if (PIN_Init(argc, argv)) return Usage();
    OutFile.open(KnobOutputFile.Value().c_str());
    // 通过使用 INS_AddInstrumentFunction 注册一个插桩函数, 在原始程序的每条指令被执行前,
    都会进入 Instruction 这个函数中
    INS_AddInstrumentFunction(Instruction, 0);
    //注册退出回调函数, 退出时调用该函数
    PIN_AddFiniFunction(Fini, 0);
    // 使用函数 PIN_StartProgram 启动程序
    PIN_StartProgram();
    return 0;
}
```

2. 2>插桩对象的创建和编译

在 testcpp 文件夹中创建 FirstCPP.c 文件作为测试对象, 写入简单的 hello world 程序:



在该文件处打开终端, 用 gcc 进行编译, 生成了编译后文件 First, 如下图所示:



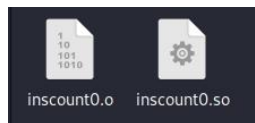
2.3>inscout0 的编译和使用

打开文件夹 ManualExamples, 输入命令行 `make inscount0.test TARGET=intel64` 进行编译:

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)~[/桌面/testcpp]
$ gcc -o First FirstCpp.c
(kali@kali)~[/桌面/testcpp]
$

zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)~[/桌面/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/ManualExamples]
$ make inscount0.test TARGET=intel64
mkdir -p obj-intel64/
g++ -Wall -Werror -Wno-unknown-pragmas -DPIN_CRT=1 -fno-stack-protector -fno-exceptions -funwind-tables -fasynchronous-unwind-tables -fno-rtti -DTARGET_IA32E -DHOST_IA32E -fPIC -DTARGET_LINUX -fabi-version-2 -fataligned-new -I../..../source/include/pin -I../..../source/include/pin/gen -isystem /home/kali/桌面/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/cxx/include -isystem /home/kali/桌面/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/crt/include -isystem /home/kali/桌面/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/crt/include/arch-x86_64 -isystem /home/kali/桌面/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/crt/include/kernel/uapi -isystem /home/kali/桌面/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/crt/include/kernel/uapi/asm-x86 -I../..../extras/components/include -I../..../extras/xed-intel64/include/xed -I../..../source/tools/Utils -I../..../source/tools/InstLib -O3 -fomit-frame-pointer -fno-strict-aliasing -Wno-dangling-pointer -c -o obj-intel64/inscount0.o inscount0.cpp
g++ -shared -Wl,-hash-style=sysv ../..../intel64/runtime/pincrt/crtbegin5.o -Wl,-Bsymbolic -Wl,-version-script=../..../source/include/pin/pintool.ver -fabi-version=2 -o obj-intel64/inscount0.so obj-intel64/inscount0.o -L../..../intel64/runtime/pincrt -L../..../intel64/lib -L../..../intel64/lib-ext -L../..../extras/xed-intel64/lib -lpin -lxd ../..../intel64/runtime/pincrt/crtend5.o -lpwindwarf -ldwarf -ldl -dynamic -nostdlib -lc++ -lc++abi -lm -dynamic -lc -dynamic -lunwind-dynamic
make -C ../..../source/tools/Utils dir obj-intel64/cp-pin.exe
make[1]: 进入目录 "/home/kali/桌面/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/Utils"
mkdir -p obj-intel64/
g++ -DTARGET_IA32E -DHOST_IA32E -DFUND_TC_TARGETCPU=FUND_CPU_INTEL64 -DFUND_TC_HOSTCPU=FUND_CPU_INTEL64 -DTARGET_LINUX -DFUND_TC_TARGETOS=FUND_OS_LINUX -DFUND_TC_HOSTOS=FUND_OS_LINUX -I../..../source/tools/Utils -O3 -std=c++11 -o obj-intel64/cp-pin.exe cp-pin.cpp -no-pie
make[1]: 离开目录 "/home/kali/桌面/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/Utils"
../..../pin -t obj-intel64/inscount0.so -- ../..../source/tools/Utils/obj-intel64/cp-pin.exe makefile obj-intel64/inscount0.makefile.copy \
> obj-intel64/inscount0.out 2>&1
cmp makefile obj-intel64/inscount0.makefile.copy
rm obj-intel64/inscount0.makefile.copy
rm obj-intel64/inscount0.out
```

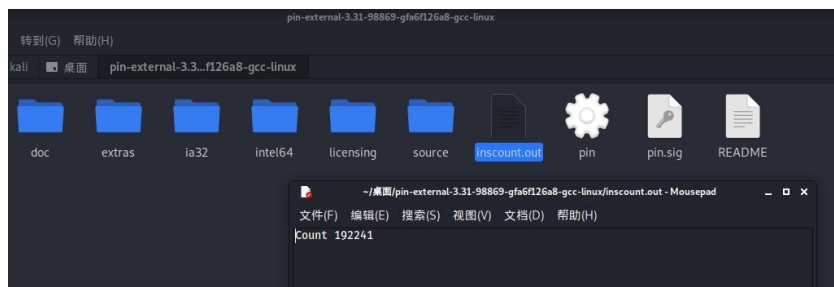
发现文件夹中出现了编译完成的文件 `inscount0.so`, 编译成功。



接下来使用这个动态链接库对 First 文件进行插桩实验, 在 pin 文件夹中打开终端, 输入命令行 `./pin -t ./source/tools/ManualExamples/obj-intel64/inscount0.so -- ../testcpp/First`, 输出如下:

```
kali@kali:~/桌面/pin-external-3.31-98869-gfa6f126a8-gcc-linux
文件 动作 编辑 查看 帮助
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)~[/桌面/pin-external-3.31-98869-gfa6f126a8-gcc-linux]
$ ./pin -t ./source/tools/ManualExamples/obj-intel64/inscount0.so -- ../testcpp/First
hello world!
(kali@kali)~[/桌面/pin-external-3.31-98869-gfa6f126a8-gcc-linux]
$
```

可见输出了 `hello world`, 再打开 pin 文件夹发现多出了一个输出文件 `inscount.out`, 文件内容如下, 即对指令数进行了插桩。



3. malloctrace 的分析和使用

malloctrace 是一种常用的 PinTool, 用于记录 malloc 和 free 的调用情况。

按照实验要求, 打开 `source/tools/ManualExamples`, 打开 `malloctrace.cpp` 查看源码:

```

std::ofstream TraceFile;
Knob< string > KnobOutputFile(KNOB_MODE_WRITEONCE, "pintool", "o", "malloctrace.out",
"specify trace file name");
VOID Arg1Before(CHAR* name, ADDRINT size) { TraceFile << name << "(" << size << ")" << endl; }
VOID MallocAfter(ADDRINT ret) { TraceFile << " returns " << ret << endl; }
VOID Image(IMG img, VOID* v) {RTN mallocRtn = RTN_FindByName(img, MALLOC);
if (RTN_Valid(mallocRtn)) {
    RTN_Open(mallocRtn);
    RTN_InsertCall(mallocRtn, IPOINT_BEFORE, (AFUNPTR)Arg1Before, IARG_ADDRINT, MALLOC,
IARG_FUNCARG_ENTRYPOINT_VALUE, 0, IARG_END);
    RTN_InsertCall(mallocRtn, IPOINT_AFTER, (AFUNPTR)MallocAfter,
IARG_FUNCRET_EXITPOINT_VALUE, IARG_END);
    RTN_Close(mallocRtn); }
RTN freeRtn = RTN_FindByName(img, FREE);
if (RTN_Valid(freeRtn)) {
    RTN_Open(freeRtn);
    RTN_InsertCall(freeRtn, IPOINT_BEFORE, (AFUNPTR)Arg1Before, IARG_ADDRINT,
FREE, IARG_FUNCARG_ENTRYPOINT_VALUE, 0, IARG_END);
    RTN_Close(freeRtn); }
}
VOID Fini(INT32 code, VOID* v) { TraceFile.close(); }
INT32 Usage() {
    cerr << "This tool produces a trace of calls to malloc." << endl;
    cerr << endl << KNOB_BASE::StringKnobSummary() << endl;
    return -1; }
int main(int argc, char* argv[]) {
    // Initialize pin & symbol manager
    PIN_InitSymbols();
    if (PIN_Init(argc, argv)) { return Usage(); }
    // Write to a file since cout and cerr maybe closed by the application
    TraceFile.open(KnobOutputFile.Value().c_str());
    TraceFile << hex;
    TraceFile.setf(ios::showbase);
    // Register Image to be called to instrument functions.
    IMG_AddInstrumentFunction(Image, 0);
    PIN_AddFiniFunction(Fini, 0);
    // Never returns
    PIN_StartProgram();
    return 0;
}

```

其中，Arg1Before 在调用 malloc 或 free 之前执行，用于记录函数名称和参数。MallocAfter 在调用 malloc 之后执行，记录返回值。Image 函数是用于在程序映像中找到并插桩 malloc 和 free 函数的代码。它会在程序加载时被调用。对于找到的 malloc 和

free 函数，使用 RTN_InsertCall 在函数调用前后插入分析函数（如 Arg1Before 和 MallocAfter）。Fini 函数在 Pin 工具完成后关闭输出文件。

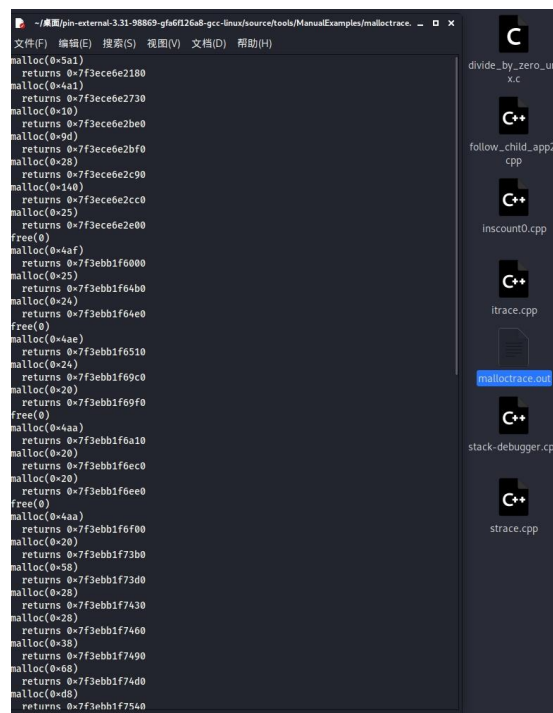
接下来，输入命令行 `make malloctrace.test TARGET=intel64`，编译运行该文件，得到.so 文件：

```
(kali@kali)~/pin-external-3.31-98869-gfa6f126a8-gcc-linux/source/tools/ManualExamples]
$ make malloctrace.test TARGET=intel64
g++ -Wall -Werror -Wno-unknown-pragmas -DPIN_CRT=1 -fno-stack-protector -fno-exceptions -funwind-tables -fasynchronous-unwind-tables
/home/kali/桌面/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/cxx/include -isystem /home/kali/桌面/pin-external-3.31-98869-
/桌面/pin-external-3.31-98869-gfa6f126a8-gcc-linux/extras/crt/include/kernel/unistd -isystem /home/kali/桌面/pin-external-3.31-98869-
s/Utils -I../source/tools/InstLib -O3 -fomit-frame-pointer -fno-strict-aliasing -Wno-dangling-pointer -c -o obj-intel64/mall
g++ -shared -Wl,-hash-style=sysv ../intel64/runtime/pincrt/crtbeginS.o -Wl,-Bsymbolic -Wl,-version-script=../source/i
../intel64/lib-ext -L../extras/xed-intel64/lib -lpin -lxd ../intel64/runtime/pincrt/crtendS.o -lpindwarf -ldwarf -ldl
../pin -t obj-intel64/malloctrace.so - ../source/tools/Utils/obj-intel64/cp-pin.exe makefile obj-intel64/malloctrace
> obj-intel64/malloctrace.out 2>&1
cmp makefile obj-intel64/malloctrace.makefile.copy
rm obj-intel64/malloctrace.makefile.copy
rm obj-intel64/malloctrace.out
```

进行插桩实验，打开生成的 out 文件，分析内容，发现其输出了 malloc 和 free 函数的具体值，如右下图所示：

前面的 malloc 是系统读入程序后进行的内存空间的分配，括号中的内容是内存空间的大小，returns 的值是内存进行存储的地址。

在使用完内存空间之后，程序会使用 free 函数进行释放。释放的时候是读入内存，进行对应位置的内存释放。



心得体会：

通过实验，具体地理解了 Pin 插桩工具的插桩框架和核心使用步骤。简单概括总结即

1. 一般的 PinTool 框架：

- 首先通过调用函数 PIN_Init 实现初始化；
- 通过使用 XXX_AddInstrumentFunction 注册一个插桩函数，在指令级插桩中，在原始程序的每条指令执行前，都会进入注册的插桩函数执行相应的操作；
- 通过使用 PIN_AddFiniFunction 注册一个程序退出时的回调函数，应用退出时会调用；
- 使用函数 PIN_StartProgram 启动程序。

2. 核心使用步骤：

- 编写要进行程序插桩测试的程序，然后编译生成可执行文件
- 找到要使用的 pintool 进行编译，生成可执行的动态链接库。
- 通过命令 `./pin -t` 命令来进行插桩实验，在对应的目录下会产生输出文件。